Version Control & Test Code Management

Day 3 · Module 3 · 2 Hours

SDET / DevOps Engineer

Intermediate

45% Theory / 55% Hands-on

TOOLS

Git

GitHub / GitLab

LEARNING OUTCOMES

What You Will Achieve Today

Manage test code effectively using Git

Apply commit hygiene, naming conventions, and repository organisation patterns specific to test automation projects.

Implement branching and merge strategies

Understand Feature Branch, GitHub Flow, and Trunk-Based Development — and how each interacts with CI pipelines for test code.

Enable collaboration between Dev and QA teams

Use pull requests, CODEOWNERS, branch protection rules, and code review practices to maintain quality in shared test repositories.

Git Workflows for Test Automation Code

Feature Branch Workflow

Use when: Mid-sized teams; structured PR review needed

1. Create feat/<ticket>-<slug> branch from main
2. Commit test code changes in small logical units
3. Open PR → CI runs → peer review → merge to main
4. Delete branch after merge; main always deployable

```
CI:  CI triggers on PR open/push; blocks merge on failure
```

GitHub Flow

Use when: Fast-moving teams; continuous deployment cadence

1. Branch from main (short-lived: hours to 1–2 days max)
2. Push frequently; open PR early as draft for visibility
3. CI runs full test suite on every push to PR branch
4. Merge to main triggers auto-deploy to staging

```
CI:  CI on every push; required status checks gate the merge button
```

Trunk-Based Development (TBD)

Use when: High-trust teams; strong automated test coverage required

1. Commit directly to main (or very short branches < 1 day)
2. Feature flags hide incomplete application functionality while changes integrate continuously
3. Every commit triggers full CI pipeline immediately
4. Team owns stability of main collectively

```
CI:  CI on every commit to main; fast suite execution is a prerequisite
```

STRATEGY COMPARISON

Choosing the Right Strategy for Test Code

Dimension	Feature Branch	GitHub Flow	Trunk-Based Dev
Branch lifespan	Days to weeks	Hours to 1–2 days	< 1 day (or direct)
CI trigger	On PR push	On PR push + merge	On every commit
Merge conflict risk	Medium–High	Low	Very Low
Review gate	Required PR review	Required PR review	Optional / pair prog
Test code stability	Good	Very Good	Can be very high with fast CI & strong coverage
Best for QA teams	Async / distributed	CI-first fast teams	High-trust senior teams

⚠ GitFlow: Less favored for CI/CD-heavy teams — better suited to release-based workflows than continuous delivery. For new automation projects following CI/CD, GitHub Flow or Trunk-Based Development are the more practical choices.

Test Code Commit Hygiene & Naming Conventions

Branch Naming Convention

Branch names — test automation repo

```
feat/QA-101-login-smoke-tests
feat/QA-204-checkout-api-tests
fix/QA-317-flaky-cart-teardown
chore/QA-400-update-testng-xml
refactor/QA-512-page-object-base

# Pattern:  <type>/<ticket>-<slug>
```

Commit Message Rules

Commit messages — Conventional Commits style

```
# Good commit — imperative, specific, atomic
feat(login): add smoke test for invalid credentials

# Good commit — references ticket
fix(cart): remove shared WebDriver state in teardown
```

Managing Test Data & Config Files

- Version-control test data (CSV, JSON) alongside test code in the same repo
- Store config.properties or pytest.ini in repo — inject env-specific values via CI env-vars
- Use .gitignore to exclude: IDE files, /target, /__pycache__, test-output/, *.log
- Never commit credentials, tokens, or PII — use GitHub Secrets or a vault
- Never commit large binary test fixtures directly — use Git LFS or external storage
- Tag test repo releases to align with app releases: git tag v1.4.2-tests

Pull Request Best Practices for Test Code

Open Draft PR → CI Runs on Push → Peer Review → Address Feedback → CI Green + Approve → Merge & Delete Branch

Keep PRs small and focused

Target < 400 lines changed. One PR = one concern (new test, fix, or refactor). Large PRs slow reviews and mask defects.

Request specific reviewers — use CODEOWNERS

Create .github/CODEOWNERS to auto-assign the right engineers. Example: */tests/payment/** @qa-payments-team

Write a meaningful PR description

Include: what changed, why, which test scenarios are covered, and how to verify locally. Use PR templates (.github/PULL_REQUEST_TEMPLATE.md).

Review test logic, not just syntax

Check: Does the test verify the right thing? Is it independent? Does it clean up state? Are assertions meaningful?

Never merge a PR with failing CI

Configure branch protection to require all status checks to pass before the merge button is enabled. Treat red CI as a blocker.

Dismiss stale approvals on new pushes

Enable 'Dismiss stale pull request approvals when new commits are pushed' in branch protection so old approvals don't pass changed code.

Branch Protection Rules & CODEOWNERS

Branch Protection — main (GitHub example)

- Require pull request before merging
- Required approvals: ≥ 1
- Require status checks to pass (CI build + test)
- Require branches to be up to date before merging
- Dismiss stale reviews on new push
- Require review from Code Owners
- Avoid bypass permissions except for tightly controlled admin/emergency use

CODEOWNERS File

.github/CODEOWNERS — test repository

```
# .github/CODEOWNERS
# <path-pattern>  <owner(s)>

# All test files
src/test/**            @qa-team

# Payment tests
src/test/**/payment/** @qa-payments
```

Required Status Checks · Draft PR note

Add CI job names (e.g. build, test) as required checks — merge button stays disabled until all pass. Note: CODEOWNERS auto-requests reviewers only when a PR is marked ready for review, not while in draft.

GitLab equivalent: Code Owners + merge request approval rules.

Handling Merge Conflicts in Test Code

Common Causes in Test Repos

- ✕ Multiple SDETs updating the same Page Object class
- ✕ testng.xml / pytest.ini modified concurrently
- ✕ Shared base test class changed on two branches
- ✕ Test data files edited independently in parallel
- ✕ Long-lived feature branches drifting far from main

Resolving conflicts — rebase workflow (follow team policy)

```
# Prefer rebase for short-lived branches (linear history)
# Use merge if your team policy values explicit merge commits
git fetch origin
git rebase origin/main   # replays your commits on latest main


# When conflict markers appear:
```

Prevention Strategies

Merge or rebase from main daily

Short-lived branches and frequent sync eliminates drift. Make it a team habit — rebase every morning.

Single responsibility per Page Object

One class per UI page; aim to have one primary SDET working on a given class within a sprint where practical. Reduces the likelihood of concurrent edits to the same file.

Split testng.xml by team / module

Instead of one monolithic suite file, use multiple included suite files. Reduces contention on configuration.

Use short-lived feature branches

GitHub Flow / TBD best practice: branches older than 2 days are a conflict risk. Merge early, merge often.

DEMO

Demo: Repo Setup → Commit → PR → Merge

01 Creating Repositories for Automation Projects

- Create a new GitHub repo; initialise with README & .gitignore (Java/Python)
- Clone locally: git clone <url> && cd <repo>
- Add framework skeleton from Day 2 lab; commit initial structure
- Push to origin: git push -u origin main

02 Commit, Merge, and Pull Request Flow

- Create feature branch: git checkout -b feat/QA-101-login-smoke
- Write a test, stage & commit: git add . && git commit -m 'feat(login): add smoke test'
- Push branch: git push origin feat/QA-101-login-smoke
- Open PR on GitHub; observe CI status checks; add reviewer; merge after approval

03 Handling Merge Conflicts

- Simulate conflict: trainer edits LoginPage on main; student edits same file on branch
- git fetch origin && git rebase origin/main — observe conflict markers
- Resolve conflict in IDE; git add <file> && git rebase --continue
- Push with force (rebase): git push --force-with-lease; verify CI is green

HANDS-ON LAB

Lab: Push, Branch, Review & Merge

45% Theory | 55% Hands-on

Task 1

Remote repo with framework code

Push Automation Code to Repository

1. Initialise or clone the course Git repository
2. Copy Day 2 framework skeleton into the repo
3. Stage and commit: git add . && git commit -m 'chore: add automation framework skeleton'
4. Push to remote: git push origin main

Task 2

Merged PR visible in repo history

Create and Merge Feature Branches

1. Create: git checkout -b feat/QA-101-login-test
2. Add a new test class; commit with Conventional Commit message
3. Push branch and open a Pull Request on GitHub/GitLab
4. Swap with a partner: review their PR, leave a comment, approve, and merge

Task 3

Resolved PR with review comments

Apply Code Review Feedback

1. Review trainer's PR — find 1 issue (missing assertion, hardcoded URL, etc.)
2. Leave an inline comment requesting a change on GitHub
3. Author addresses feedback, pushes a new commit to the PR branch
4. Re-review, approve, and verify CI is green before merging

Day 3 Deliverables & Key Takeaways

Deliverables You Take Away

- Git Repository with test automation code pushed to GitHub/GitLab
- Branching & Versioning Strategy Document for the test repo

Key Takeaways

1. GitHub Flow and Trunk-Based Development are the preferred strategies for CI-first test automation
2. Short-lived branches and frequent merges prevent the merge conflicts that plague long-lived test branches
3. Conventional Commit messages and consistent branch naming make test repo history navigable
4. Branch protection rules + CODEOWNERS are the primary quality gates for shared test repositories
5. Never commit credentials or PII — use GitHub Secrets / vault and inject via CI env-vars

Tomorrow — Day 4: CI Pipeline Design with Automated Testing · Jenkins / GitHub Actions / Azure DevOps pipeline config

Hands-on: Create a CI pipeline · add test execution steps · trigger on commit · analyse failed builds

RECAP QUIZ

Module 3 – Quick Knowledge Check

Q1. What is the main risk of long-lived feature branches in a test automation repo?

- The branch drifts far from main, increasing merge conflict probability and delaying integration. The longer the branch, the larger and more complex the eventual merge — and the more likely CI regressions are missed.

Q2. What does git push --force-with-lease do, and why is it safer than --force?

- --force-with-lease only overwrites the remote branch if no one else has pushed to it since your last fetch. It prevents accidentally overwriting a teammate's commits, making it the safe default for post-rebase pushes.

Q3. What two things does a .github/CODEOWNERS file control?

- 1) It automatically assigns specific reviewers (users or teams) when a PR modifies files matching a path pattern. 2) When 'Require review from Code Owners' is enabled in branch protection, it blocks merges until the assigned owner approves.

Q4. What are the advantages of git rebase over git merge when syncing a feature branch, and when might a team choose merge instead?

- Rebase replays your commits on top of the latest main, producing a linear history without merge commits — easier to read, bisect, and blame. Teams may prefer merge commits for auditability or when explicit integration points matter. Follow your team's documented policy.